Challenge Questions

Authors:

- Seyedmohammad Tabatabaei 10967133
- Mostafa Moazenkakhki 11033591

Table of Contents

1. Unsuccessful Confirmable PUTs to Local CoAP Server
2. Equal Confirmable vs Non-confirmable GETs to coap.me Resources
3. MQTT Clients Subscribing with Multi-level Wildcards
4. MQTT Will Messages to Topics Starting with "university"
5. MQTT Subscribers Receiving Will Messages (No Wildcard Subscriptions)
6. MQTT Retain Publishes with QoS "At most once" to Mosquitto Broker
7. MQTT-SN Messages Sent to Local Broker on Port 1885
8. Calculations for the Second Excercise (the explanations are available in Excercise 2 file)

🧠 CQ1) How many different Confirmable PUT requests obtained an unsuccessful response from the local CoAP server?

The provided code determines the number of different **Confirmable (CON) PUT requests** that received an **unsuccessful response** from a local CoAP server.

The code iterates over all captured packets and processes those containing CoAP data.
For each packet, it extracts important attributes such as:

- **CoAP message ID (MID)**
- **Source and destination IP addresses**
- **UDP ports**
- **CoAP message type**
- **CoAP code**

🔍 Identification of a Confirmable PUT Request

A Confirmable PUT request is identified when:

- The **CoAP type** (`coap_type`) is `0` (Confirmable message)
- The **CoAP code** (`coap_code`) is `3` (indicating a PUT request)
- The packet is destined for the **server's listening port** (`dst_port == server_port`)
- The **destination IP address** is either `127.0.0.1 (localhost)` or `10.0.2.15`

When a Confirmable PUT request is detected, its **MID** is stored in the `requests_1` dictionary with fields indicating:

- Whether an **acknowledgment (ACK)** was received
- Whether the **request was successful**

📥 Monitoring ACK Responses

Subsequently, the code monitors for **Empty ACK messages** from the server:

- If an **ACK** with a **CoAP code of 128 or higher** is received (typically indicating an error response), the corresponding request is marked as **unsuccessful**
- If an **ACK** with a **CoAP code less than 128** is received (indicating success), only the reception of the ACK is noted

---

📊 Final Calculation

Finally, the number of **unsuccessful Confirmable PUT requests** is computed by counting all requests in `requests_1` where:

```
successful flag == False
```

```
!pip install pyshark
!pip install nest_asyncio
!apt-get install tshark
import pyshark
import nest_asyncio
nest_asyncio.apply()
```

Show hidden output

```
import pyshark

pcap_file = '/content/challenge2.pcapng'

server_port = 5683   # default server port for CoAP

capture = pyshark.FileCapture(pcap_file, display_filter="coap") # getting the coap messages from the capture file
```

```
from collections import defaultdict

# Dictionary to store request MIDs
requests_1 = {}
uri_paths = {}


for packet in capture:
    try:
        coap_layer = packet.coap
        udp_layer = packet.udp

        # Extract key values
        mid = coap_layer.mid  # Message ID
        src_ip = packet.ip.src
        dst_ip = packet.ip.dst
        src_port = int(udp_layer.srcport)
        dst_port = int(udp_layer.dstport)
        coap_type = int(coap_layer.type)
        coap_code = int(coap_layer.code)  # Request/Response Code

        # First Question code
        # Identify Confirmable PUT Requests (Client → Server)
        if coap_type == 0 and coap_code == 3 and dst_port == server_port and (dst_ip in ["127.0.0.1", "10.0.2.15"]):
            requests_1[mid] = {"src": src_ip, "dst": dst_ip, "ack_received": False, "successful": True}

        # Identify Empty ACKs (Server → Client)
        elif coap_type == 2 and coap_code >= 128 :
            if mid in requests_1:
                requests_1[mid]["ack_received"] = True
                requests_1[mid]["successful"] = False
        elif coap_type == 2 and coap_code < 128:
            if mid in requests_1:
                requests_1[mid]["ack_received"] = True
```

```
        # Second question 2
        if dst_ip == '134.102.218.18' and coap_code == 1:
          uri_path = coap_layer.opt_uri_path_recon.showname_value
          token = coap_layer.token
          if uri_path not in uri_paths:
              uri_paths[uri_path] = [{"mid": mid, "token": token, "type": coap_type}]
          else:
              uri_paths[uri_path].append({"mid": mid, "token": token, "type": coap_type})




    except AttributeError:
        continue

capture.close()

num_received_ack = sum(1 for ack in requests_1.values() if ack["ack_received"])
num_unsuccessful = sum(1 for ack in requests_1.values() if not ack["successful"])
```

```
num_received_ack = sum(1 for ack in requests_1.values() if ack["ack_received"])
num_unsuccessful = sum(1 for ack in requests_1.values() if not ack["successful"])
# Print results
print(f"Total Confirmable PUT requests: {len(requests_1)}")
print(f"Total Confirmable messages that received an Ack: {num_received_ack}")
print(f"Total unsuccessful responses: {num_unsuccessful}")
```

```
Total Confirmable PUT requests: 26
Total Confirmable messages that received an Ack: 26
Total unsuccessful responses: 22
```

🔄 CQ2) How many CoAP resources in the coap.me public server received the same number of unique Confirmable and Non-Confirmable GET requests?

> Assuming a resource receives **X** different CONFIRMABLE requests and **Y** different NONCONFIRMABLE GET requests,
> how many resources have **X = Y**, with **X > 0**?

📍 Resources with Equal Unique Confirmable and Non-Confirmable GET Requests to coap.me

The second analysis focused on the **coap.me public CoAP server**.
The goal was to determine how many CoAP resources received the **same number of unique Confirmable and Non-Confirmable GET requests**,
with **both counts being greater than zero**.

🔎 Analysis Process

- The code groups all captured **GET requests** based on their **URI path**.
- For each resource:
  - **Unique Confirmable GET requests** are identified using their **tokens and message IDs**
    when the type is **Confirmable** (`coap_type == 0`).
  - **Unique Non-Confirmable GET requests** are similarly identified
    when the type is **Non-Confirmable**.

🧮 Result Calculation

- For each URI path, the counts of **unique Confirmable** and **Non-Confirmable** requests are stored.
- After processing, resources where the number of unique Confirmable and Non-Confirmable GETs are **equal and greater than zero** (**X = Y**, **X > 0**) are **filtered**.
- The final result includes:
  - The **number of such resources**
  - A **detailed list of matching resources**

```
uri_paths_stats = {}

for uri_path, requests in uri_paths.items():
    unique_con_tokens = set()
    unique_non_tokens = set()
    unique_con_mids = set()
    unique_non_mids = set()

    for req in requests:
        if req["type"] == 0:  # Confirmable GET
            if req["token"]:
                unique_con_tokens.add(req["token"])
            if req["mid"]:
                unique_con_mids.add(req["mid"])
        else:  # Non-Confirmable GET
            if req["token"]:
                unique_non_tokens.add(req["token"])
            if req["mid"]:
                unique_non_mids.add(req["mid"])

    uri_paths_stats[uri_path] = {
        "num_unique_con_get": len(unique_con_tokens),  # Unique CON GETs
        "num_unique_non_get": len(unique_non_tokens),  # Unique NON GETs
    }

# Count how many resources have X (unique CON GETs) == Y (unique NON GETs) and X > 0
matching_resources = {uri: stats for uri, stats in uri_paths_stats.items()
                      if stats["num_unique_con_get"] == stats["num_unique_non_get"] and stats["num_unique_con_get"] > 0}

# Output the result
print(f"Total resources where X = Y and X > 0: {len(matching_resources)}")
print("Matching resources:", matching_resources)
```

```
Total resources where X = Y and X > 0: 3
Matching resources: {'/large': {'num_unique_con_get': 1, 'num_unique_non_get': 1}, '/validate': {'num_unique_con_get': 1, 'num_unique_nc
```

🔔 CQ3) How many different MQTT clients subscribe to the public broker HiveMQ using multi-level wildcards?

The third question involved analyzing **MQTT traffic** to determine how many different **MQTT clients** subscribed to the public **HiveMQ broker** using **multi-level wildcards** in their subscription topics.

🔍 Packet Filtering and Wildcard Detection

- The code filters packets to only those directed at the **HiveMQ broker's IP address**: `18.192.151.104`.
- It specifically checks for **SUBSCRIBE messages** (message type `8`).
- For each subscription, the **topic field** is analyzed.
- If the topic contains the **multi-level wildcard character (`#`)**, the **client is recorded**.

📊 Result Compilation

- **Unique clients** subscribing with multi-level wildcards are tracked in a **set** to ensure **no duplicates**.
- The final result is the **count of unique MQTT clients** who used **multi-level wildcards** in their subscriptions.

```
import pyshark

# Path to PCAP file
pcap_path = '/content/challenge2.pcapng'

capture = pyshark.FileCapture(pcap_path, display_filter="mqtt")

multi_wildcard_clients = set()

for pkt in capture:
    try:
        if pkt.ip.dst != "18.192.151.104":
            continue
```

```
        mqtt = pkt.mqtt

        # look for SUBSCRIBE messages (type 8)
        if int(mqtt.msgtype) != 8:
            continue

        # Check for "#" in the subscription topic
        topic = mqtt.topic
        if "#" in topic:
            # Use source IP and port tuple to uniquely identify the client
            client_key = (pkt.ip.src, pkt.tcp.srcport)
            multi_wildcard_clients.add(client_key)

    except AttributeError:
        continue

# Output the correct result
print(f"Total unique MQTT clients using multi-level wildcards: {len(multi_wildcard_clients)}")
```

```
Total unique MQTT clients using multi-level wildcards: 4
```

🏫 CQ4) How many different MQTT clients specify a Last Will Message to be directed to a topic having as first level "university"?

This part focused on identifying how many different **MQTT clients** specified a **Last Will message** directed to a topic starting with `"university/"`.

🔍 Packet Filtering Criteria

- The code filters the capture to only **MQTT CONNECT packets** (`msgtype == 1`).
- It checks if the **`willtopic`** field matches the pattern beginning with `"university/"`.

📊 Result Compilation

- Each client that fulfills this condition is collected into a **set** to ensure **uniqueness**.
- The final count represents the number of **unique clients** that configured a **Last Will message** targeted at such topics.

```
import pyshark

university_will_clients = set()

# Path to the uploaded PCAP file
pcap_file = "/content/challenge2.pcapng"

# Read MQTT packets with a filter for CONNECT messages
capture = pyshark.FileCapture(pcap_file, display_filter='mqtt && mqtt.msgtype == 1 && mqtt.willtopic matches "^university/"')

# Process each packet
for packet in capture:
    try:
        mqtt_layer = packet.mqtt
        will_topic = mqtt_layer.get("mqtt.willtopic", None)
        # Ensure topic starts with "university/"
        if will_topic.startswith("university/"):
          university_will_clients.add(mqtt_layer)

    except AttributeError:
        continue  # Skip packets without MQTT data

# Output the results
print(f"Total unique MQTT clients using a Last Will topic starting with 'university/': {len(university_will_clients)}")
```

```
Total unique MQTT clients using a Last Will topic starting with 'university/': 1
```

📍 CQ5) How many MQTT subscribers receive a Last Will message derived from a subscription without a wildcard?

This section investigates how many **MQTT subscribers**, who subscribed to topics **without using wildcards**, would end up receiving a **Last Will message**.

🛠️ Approach

1. **Filter all MQTT SUBSCRIBE packets** that do **not** contain a **multilevel wildcard (`#`)** to collect their **topics**.
2. **Gather all CONNECT packets** that include a **Last Will message**, and extract the associated **topics**.
3. **Count how many subscriber topics** exactly **match** any of the topics designated in a **Last Will message**.

📊 Final Result

The result is the **number of subscribers** (without using wildcards) who would receive a **Last Will message** based on a **topic match**.

```
# we save the topics that these subscribers are subscribed to
subscribers_topics = []

# then we save the topics that have been sent back a wildcard and compare them
# to the subscribers topics. All the subcribers of that topic will receive a
# last will message
last_will_topics = []

capture1 = pyshark.FileCapture(pcap_file, display_filter='mqtt && mqtt.msgtype == 8 && !(mqtt.topic contains "#") && !(mqtt.topic contains "
capture2 = pyshark.FileCapture(pcap_file, display_filter='mqtt && mqtt.msgtype == 1 && mqtt.willtopic')

for packet in capture1:
    try:
        mqtt_layer = packet.mqtt
        topic = mqtt_layer.topic
        subscribers_topics.append(topic)

    except AttributeError:
        continue

for packet in capture2:
    try:
        mqtt_layer = packet.mqtt
        willtopic = mqtt_layer.willtopic
        last_will_topics.append(will_topic)

    except AttributeError:
        continue

def count_matches(list1, list2):
    set2 = set(list2)

    count = sum(1 for item in list1 if item in set2)

    return count

print('MQTT subscribers receive a last will message derived from a subscription without a wildcard: ', count_matches(subscribers_topics, las
```

```
MQTT subscribers receive a last will message derived from a subscription without a wildcard:  3
```

CQ6) How many MQTT publish messages directed to the public broker mosquitto are sent with the retain option and use QoS “At most once”?

📦 CQ6) How many MQTT publish messages directed to the public broker Mosquitto are sent with the retain option and use QoS “At most once”?

This part analyzes how many **MQTT PUBLISH messages** sent to the public **Mosquitto broker** (`5.196.78.28`) meet **two specific conditions**:

✅ Conditions

1. The **retain flag** is set (`retain = 1`), meaning the broker should **store the last message** on the topic.
2. The **Quality of Service (QoS) level** is `0` (**"At most once"**), meaning **delivery is not guaranteed**.

📊 Result Calculation

- The code **filters packets** matching both criteria.
- It then **counts** them.
- The final count represents how many such **retained messages with QoS 0** were directed to **Mosquitto**.

```
mosquitto_ip = "5.196.78.28"

capture_5 = pyshark.FileCapture(pcap_file, display_filter=f"mqtt && mqtt.msgtype == 3 and ip.dst == {mosquitto_ip} && mqtt.retain == 1 && mc
count= 0
for packet in capture_5:
    count+=1
print(count)
```

```
208
```

CQ7) How many MQTT-SN messages on port 1885 are sent by the clients to a broker in the local machine?

We first use **mqtt-sn** to filter out the packets using this protocol in order to analyze them.
However, as seen in the code, there are **no messages sent with this protocol** in the provided capture file to analyze.

```
capture = pyshark.FileCapture(pcap_file, display_filter='mqtt-sn')
print('Number of packets configured with mqtt-sn is',len(capture))
```

```
Number of packets configured with mqtt-sn is 0
```

Calculations for the Second Excercise (the explanations are available in Excercise 2 file)

```
# Energy Consumption Analysis: CoAP vs. MQTT for Sensor Networks
# Authors: Seyedmohammad Tabatabaei, Mostafa Moazenkakhki

# Constants
E_Tx = 50e-9  # Transmission energy per bit (50 nJ/bit)
E_Rx = 58e-9  # Reception energy per bit (58 nJ/bit)
E_c = 2.4e-3  # Processing energy per computation (2.4 mJ)
hours = 24    # Time period (24 hours)
interval = 30 # Data transmission interval (minutes)

# Calculate number of transactions
transactions = (hours * 60) / interval

# Protocol message sizes (in bytes)
message_sizes = {
    'coap': {
        'GET_req': 60,
        'GET_resp': 55,
    },
    'mqtt': {
        'Connect': 54,
        'Connect_Ack': 47,
        'Subscribe': 58,
        'Sub_Ack': 52,
        'Publish': 68,
    }
}

# Convert bytes to bits
for protocol in message_sizes:
    for msg in message_sizes[protocol]:
        message_sizes[protocol][msg] *= 8  # 1 byte = 8 bits

# CoAP Energy Calculations
def calculate_coap_energy():
    # Sensor energy per transaction
    sensor_per_tx = (message_sizes['coap']['GET_req'] * E_Rx) + (message_sizes['coap']['GET_resp'] * E_Tx)
    sensor_total = sensor_per_tx * transactions

    # Value node energy per transaction
    value_per_tx = (message_sizes['coap']['GET_req'] * E_Tx) + (message_sizes['coap']['GET_resp'] * E_Rx) + E_c
    value_total = value_per_tx * transactions

    return {
```

```
        'sensor': sensor_total * 1e3,  # Convert to mJ
        'value': value_total * 1e3,     # Convert to mJ
        'total': (sensor_total + value_total) * 1e3
    }

# MQTT Energy Calculations
def calculate_mqtt_energy():
    # Value node energy
    connect_part = (message_sizes['mqtt']['Connect'] * E_Tx) + (message_sizes['mqtt']['Connect_Ack'] * E_Rx)
    subscribe_part = (message_sizes['mqtt']['Subscribe'] * E_Tx) + (message_sizes['mqtt']['Sub_Ack'] * E_Rx)
    publish_part = transactions * ((message_sizes['mqtt']['Publish'] * E_Rx) + E_c)
    value_total = connect_part + subscribe_part + publish_part

    # Sensor energy
    sensor_connect = (message_sizes['mqtt']['Connect'] * E_Tx) + (message_sizes['mqtt']['Connect_Ack'] * E_Rx)
    sensor_publish = transactions * (message_sizes['mqtt']['Publish'] * E_Tx)
    sensor_total = sensor_connect + sensor_publish

    # Raspberry PI energy
    connect_part_pi = 2 * (message_sizes['mqtt']['Connect'] * E_Tx) + (message_sizes['mqtt']['Connect_Ack'] * E_Rx)
    subscribe_part_pi = (message_sizes['mqtt']['Subscribe'] * E_Tx) + (message_sizes['mqtt']['Sub_Ack'] * E_Rx)
    publish_part_pi = 2 * transactions * (message_sizes['mqtt']['Publish'] * E_Rx)
    raspberry_total = connect_part_pi + subscribe_part_pi + publish_part_pi

    return {
        'sensor': sensor_total * 1e3,  # Convert to mJ
        'value': value_total * 1e3,     # Convert to mJ
        'raspberry': raspberry_total * 1e3,
        'total': (sensor_total + value_total + raspberry_total) * 1e3
    }

# MQTT-SN Energy Calculations (estimated 30% reduction)
def calculate_mqtt_sn_energy(mqtt_energy):
    return {
        'sensor': mqtt_energy['sensor'] * 0.7,
        'value': mqtt_energy['value'] * 0.7,
        'raspberry': mqtt_energy['raspberry'] * 0.7,
        'total': mqtt_energy['total'] * 0.7
    }

# Calculate all energies
coap_energy = calculate_coap_energy()
mqtt_energy = calculate_mqtt_energy()
mqtt_sn_energy = calculate_mqtt_sn_energy(mqtt_energy)

# Print results
print("Energy Consumption Results (in mJ over 24 hours)")
print("\nCoAP Protocol:")
print(f"- Sensor: {coap_energy['sensor']:.3f} mJ")
print(f"- Value Node: {coap_energy['value']:.3f} mJ")
print(f"- Total: {coap_energy['total']:.3f} mJ")

print("\nMQTT Protocol (QoS 0):")
print(f"- Sensor: {mqtt_energy['sensor']:.3f} mJ")
print(f"- Value Node: {mqtt_energy['value']:.3f} mJ")
print(f"- Raspberry PI: {mqtt_energy['raspberry']:.3f} mJ")
print(f"- Total: {mqtt_energy['total']:.3f} mJ")

print("\nMQTT-SN Protocol (Estimated 30% reduction):")
print(f"- Sensor: {mqtt_sn_energy['sensor']:.3f} mJ")
print(f"- Value Node: {mqtt_sn_energy['value']:.3f} mJ")
print(f"- Raspberry PI: {mqtt_sn_energy['raspberry']:.3f} mJ")
print(f"- Total: {mqtt_sn_energy['total']:.3f} mJ")

# Comparison table
print("\nComparison Table:")
print("+-----------------+-----------+-----------+-----------+")
print("| Protocol        | Sensor    | Value Node| Total     |")
print("+-----------------+-----------+-----------+-----------+")
print(f"| CoAP            | {coap_energy['sensor']:7.3f}  | {coap_energy['value']:7.3f}  | {coap_energy['total']:7.3f}  |")
print(f"| MQTT            | {mqtt_energy['sensor']:7.3f}  | {mqtt_energy['value']:7.3f}  | {mqtt_energy['total']:7.3f}  |")
print(f"| MQTT-SN (est.) | {mqtt_sn_energy['sensor']:7.3f}  | {mqtt_sn_energy['value']:7.3f}  | {mqtt_sn_energy['total']:7.3f}  |")
print("+-----------------+-----------+-----------+-----------+")

# Packet size comparison
print("\nPacket Size Comparison:")
print("+-----------------+---------------------+")
```

```
print("| Protocol        | Approximate Size    |")
print("+-----------------+---------------------+")
print("| MQTT            | 50-100 bytes        |")
print("| MQTT-SN         | 35-50 bytes         |")
print("+-----------------+---------------------+")
```

```
Energy Consumption Results (in mJ over 24 hours)

CoAP Protocol:
- Sensor: 2.392 mJ
- Value Node: 117.577 mJ
- Total: 119.969 mJ

MQTT Protocol (QoS 0):
- Sensor: 1.349 mJ
- Value Node: 116.805 mJ
- Raspberry PI: 3.141 mJ
- Total: 121.296 mJ

MQTT-SN Protocol (Estimated 30% reduction):
- Sensor: 0.944 mJ
- Value Node: 81.764 mJ
- Raspberry PI: 2.199 mJ
- Total: 84.907 mJ

Comparison Table:
+-----------------+-----------+-----------+-----------+
| Protocol        | Sensor    | Value Node| Total     |
+-----------------+-----------+-----------+-----------+
| CoAP            |   2.392   | 117.577   | 119.969   |
| MQTT            |   1.349   | 116.805   | 121.296   |
| MQTT-SN (est.)  |   0.944   |  81.764   |  84.907   |
+-----------------+-----------+-----------+-----------+

Packet Size Comparison:
+-----------------+---------------------+
| Protocol        | Approximate Size    |
+-----------------+---------------------+
| MQTT            | 50-100 bytes        |
| MQTT-SN         | 35-50 bytes         |
+-----------------+---------------------+
```